

Sumário

Desafio Técnico — AI Engineer / Product Owner (AI)	1
Deep Seed Solutions	1
1. Por que este desafio	1
2. O problema	1
3. O que você deve construir	2
4. Spec-Driven Development (obrigatório)	2
5. Escopo — leia com atenção	2
6. Entregáveis	2
7. Como avaliamos	2
8. Prazo e formato	3
9. Etapa seguinte — entrevista técnica (45 min)	3

Desafio Técnico — AI Engineer / Product Owner (AI)

Deep Seed Solutions

TL;DR — Construa um agente em Python que ingere um pequeno corpus de engenharia offshore, monta um **knowledge graph** e responde perguntas usando **GraphRAG** (recuperação guiada pelo grafo, com travessia multi-hop e rastreabilidade). O trabalho deve ser conduzido **spec-first**: a especificação vem antes do código. Tempo-alvo: **3-4 horas**. Entrega: **repositório no GitHub + vídeo de 5 min**.

1. Por que este desafio

Na DSS, esta vaga é de um profissional híbrido: pensa produto como um PO, mas implementa como um engenheiro de IA. O dia a dia envolve dados de engenharia offshore — reservatório, poços, flow assurance, topside, economia — disciplinas que se referenciam o tempo todo: uma decisão em uma muda uma restrição em outra, que muda uma premissa de custo em uma terceira. Dados assim são, por natureza, um **grafo**.

Este desafio simula uma fatia mínima desse mundo, com um corpus genérico e fictício. Não testamos quanto de offshore você já sabe — o domínio está explicado no material. Testamos como você **modela conhecimento como grafo**, como desenha **recuperação multi-hop** e o quanto você é disciplinado em **Spec-Driven Development** e em Python.

2. O problema

Você recebe um corpus pequeno (repo/data/corpus/) com ~10 documentos de um estudo fictício de seleção de concepção de um campo offshore (“Campo Açu-Deep”). Os documentos se referenciam entre si: uma decisão de **diâmetro de flowline** no documento de Subsea afeta uma **restrição de flow assurance**, que por sua vez muda uma premissa de **CAPEX** no documento de Economics — e assim por diante.

Algumas perguntas só podem ser respondidas **conectando informações de 2-3 documentos diferentes**. É exatamente aí que um RAG ingênuo (só embeddings + top-k) costuma falhar e onde o GraphRAG brilha.

Atenção — o corpus tem armadilhas de propósito: ele contém mais de um cenário de desenvolvimento (com vocabulário muito parecido) e uma decisão que passou por revisão. Nem todo documento que “parece” relevante é a fonte **vigente**. Parte do desafio é recuperar a informação certa, não a mais parecida.

3. O que você deve construir

Um agente em Python que:

1. **Ingestão → grafo.** Lê o corpus e constrói um knowledge graph com entidades tipadas (ex.: Projeto, Cenário, Conceito, Sistema, Equipamento, Restrição, Decisão, PremissaEconômica) e arestas tipadas (ex.: IMPACTA, DERIVA_DE, PERTENCE_A, RESTRINGE). A estratégia de extração é sua escolha (LLM, regras, híbrido) — **justifique-a na SPEC**.
2. **GraphRAG.** Dada uma pergunta em linguagem natural, recupera o subgrafo relevante, faz a **travessia multi-hop** necessária e gera uma resposta ancorada nas evidências, **citando os documentos/nós de origem**.
3. **Responde as perguntas-ouro.** Em `repo/data/questions.yaml` há 5 perguntas; pelo menos 3 exigem multi-hop. Seu agente deve respondê-las com rastreabilidade.

Você pode usar qualquer biblioteca (`networkx`, `pydantic`, um cliente de LLM, etc.). Se usar um LLM, deixe a chave configurável por variável de ambiente e **garanta um modo de fallback** (ex.: extração por regras) para rodarmos sem sua chave — ou documente claramente como executar.

4. Spec-Driven Development (obrigatório)

Antes de codar, preencha `repo/SPEC.md` (há um esqueleto). A SPEC deve definir, no mínimo: o **schema do grafo** (tipos de nó e aresta e por quê), a **estratégia de recuperação** (como decide o ponto de entrada e como faz a travessia), e a **forma de avaliação** (como você sabe que acertou). Queremos ver que a spec **guiou** o código — não que foi escrita depois para justificá-lo. Commits incrementais que mostrem SPEC → código → teste são um forte sinal positivo.

5. Escopo — leia com atenção

O escopo é **deliberadamente maior do que cabe em 3-4 horas**. Não tente fazer tudo. Parte do que avaliamos é **o que você decide cortar** e por quê. Um “vertical slice” estreito que funciona ponta-a-ponta vale muito mais do que três módulos pela metade. Liste no README o que ficou de fora e como você abordaria se tivesse mais tempo.

Sugestão de corte sensato: schema enxuto (4-6 tipos de nó), extração simples mas explicada, GraphRAG que cobre bem 1-2 das perguntas multi-hop ponta-a-ponta, com testes. Não precisa de UI, não precisa de banco de grafo (um `networkx` em memória basta), não precisa fine-tuning.

Regra anti-atalho (importante): trate o corpus como se fossem **10.000 documentos**. Jogar todos os documentos de uma vez no contexto de um LLM e deixar ele “se virar” é **reprovação automática** — não demonstra recuperação. A sua recuperação precisa ser **seletiva** (traz só o subgrafo relevante) e **explicável** (você sabe dizer por que trouxe aqueles nós e não outros).

6. Entregáveis

1. **Repositório GitHub** (público ou privado com acesso a @deepseedsolutions) contendo:
 - SPEC.md preenchido.
 - Código (`src/`), com README.md explicando arquitetura, decisões e trade-offs.
 - Testes (`tests/`) — não exaustivos, mas mostrando que você testa o que importa.
 - Instruções de execução (`make run` ou `python -m graphrag ...`).
2. **Vídeo de até 5 minutos** (Loom/qualquer link) percorrendo seu raciocínio: por que o schema, como o GraphRAG resolve o multi-hop, e o que cortou. **O vídeo é essencial** — é onde vemos que você entende o que entregou.

7. Como avaliamos

Eixo	Peso	O que procuramos
Modelagem do grafo de conhecimento	25%	Schema bem pensado, relações tipadas com propósito, multi-hop possível
GraphRAG / recuperação	25%	Travessia real do grafo (não só top-k de embeddings), rastreabilidade, lida com multi-hop
Spec-Driven Development	20%	A spec guiou o código; clareza; coerência entre SPEC, código e testes
Qualidade de engenharia (Python)	20%	Estrutura, tipagem, testes, legibilidade, execução reproduzível
Pensamento de produto	10%	Decisões de escopo, trade-offs explícitos, comunicação no README e no vídeo

Não procuramos um produto polido. Procuramos profundidade técnica real e clareza de raciocínio. Honestidade sobre limitações conta a favor.

8. Prazo e formato

- Esforço-alvo: **3-4 horas**. Janela de entrega: **5 dias corridos** a partir do recebimento (flexível — combine conosco).
- Use IA assistente à vontade (Copilot, Claude, etc.) — usamos no dia a dia. Mas esteja preparado para **defender cada decisão** na entrevista técnica.

9. Etapa seguinte — entrevista técnica (45 min)

Conversa ao vivo sobre o que você construiu. O roteiro está fora deste documento, mas espere perguntas como: “se o corpus tivesse 10.000 documentos, o que muda no seu GraphRAG?”, “por que hipergrafo não, ou por que sim?”, “como você avaliaria a qualidade das respostas em escala?”. A ideia não é pegar você — é ver até onde vai a profundidade.

Dúvidas: ricardo.brigante@deepseedsolutions.com